



CSE332.2

Computer Organization & Architecture

Project: MIPS CPU & OS Design

Project Report

SUBMITTED to,

Faculty: MAQm

Section: 2

SUBMITTED by,

Group - 7
2221080642 - Tashfiq Mahmud Niloy
2312187642 - Touhid Ara Himu
2221838642 - Ihtishamul Haq Shimanto
2232858642 - Farhan Fuad Abir

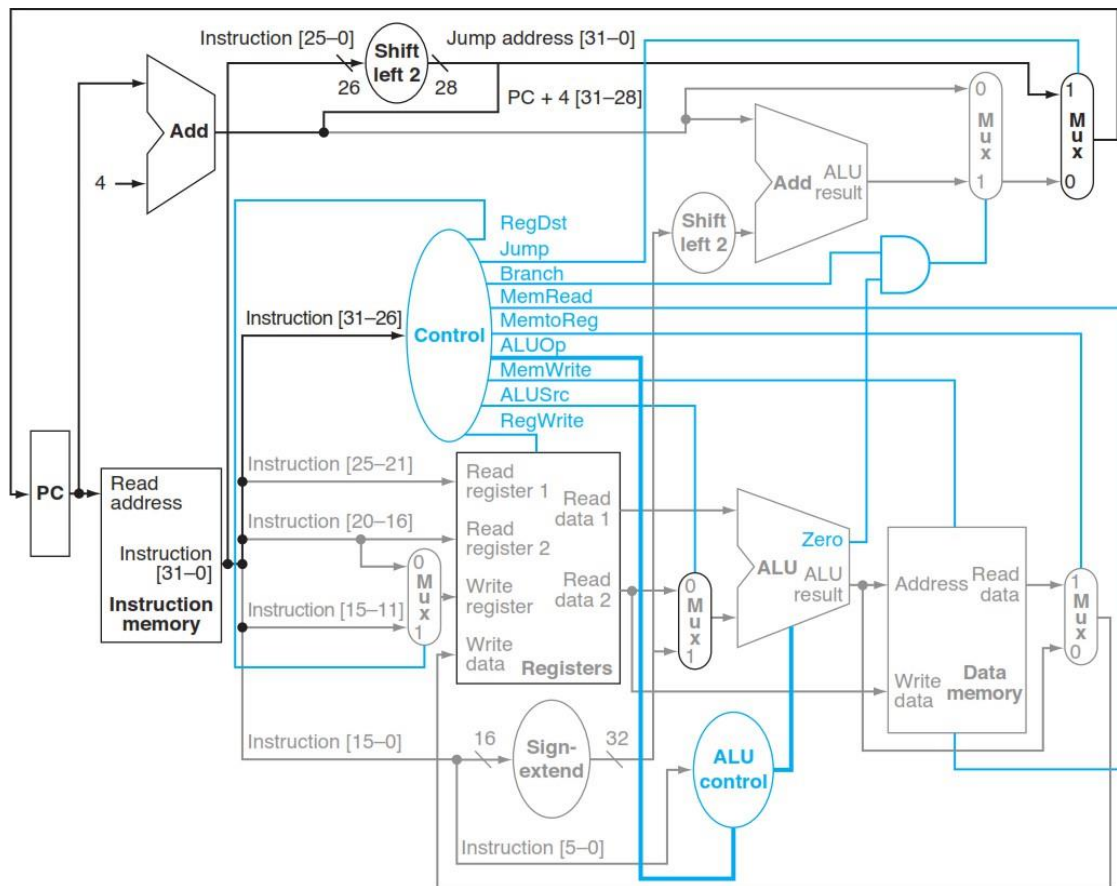
Submission Date: 31 AUGUST

Semester: Summer 2025

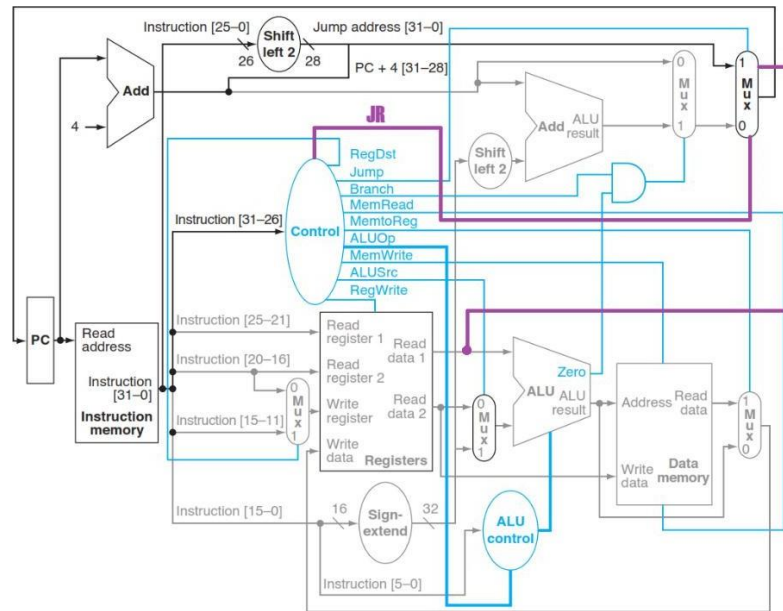
North South University

Introduction to MIPS CPU & OS

The MIPS (Microprocessor without Interlocked Pipeline Stages) architecture represents a simplified and clean implementation of the Reduced Instruction Set Computer (RISC) design philosophy. As a widely studied and implemented architecture, MIPS provides a clear framework for understanding fundamental processor design concepts, including instruction formats, control signal management, data path organization, and modularity in instruction execution. It uses fixed-length 32-bit instructions and a load/store architecture, ensuring consistent decoding and efficient pipelining. The MIPS instruction set is composed of three fundamental types: R-type (register), I-type (immediate), and J-type (jump). These support a wide variety of operations such as arithmetic, logic, memory access, branching, and jumping. In standard form, the MIPS processor includes instructions such as ADD, SUB, AND, OR, SLT, LW, SW, BEQ, BNE, J, and more.



At the heart of the MIPS processor lies the Control Unit, which is responsible for interpreting instruction opcodes and generating the appropriate control signals that dictate the behavior of the data path components. The Control Unit orchestrates the execution by managing elements such as the register file, ALU, data memory, and program counter (PC). It ensures that, depending on the type of instruction (R, I, or J), the relevant components are activated and data flows through



The JR instruction is an R-type instruction that performs an unconditional jump to the address contained in a specified register. It is typically used to return from a subroutine by jumping to the address in \$ra. Our modification to support JR included adding a control path in the datapath to allow the program counter to be updated from the register file, rather than using an immediate target address. The control unit was updated to recognize the function code associated with JR and to switch the PC input accordingly.

MIPS OS – Arithmetic Operations (MIN, MAX, MEAN):

Beyond these control-flow instructions, we added support for three custom arithmetic operations: MIN, MAX, and MEAN. These instructions were introduced as new R-type operations, each with a unique function code. The MIN operation compares the values of two registers and stores the lesser of the two in the destination register. The MAX operation performs the opposite, storing the greater of the two. The MEAN instruction computes the average of the two operands by summing them and performing a logical right shift by one bit (equivalent to division by two).

To support these custom instructions, the Arithmetic Logic Unit (ALU) was expanded to recognize and correctly execute these operations. This required modifying the ALU's internal case structure to detect the custom function codes and implement the corresponding logic. For MEAN, the ALU performs addition followed by a right-shift operation. For MIN and MAX, it uses comparison logic to determine the appropriate output.

Overall, the enhancements introduced in this project significantly extend the capability of the baseline MIPS processor. The addition of JAL and JR allows for more complex program flow and function structuring, bringing the architecture closer to real-world usage scenarios. Meanwhile, the introduction of the MIN, MAX, and MEAN arithmetic operations demonstrates how custom instruction sets can be integrated into the processor pipeline, providing specialized operations that improve performance and reduce software complexity.

Modifications in MIPS CPU & OS Design

Updated Control.v

```
timescale 1ns/1ns

module ControlUnit(
    input [5:0] Opcode,
    input [5:0] Func,
    input Zero,
    output reg MemtoReg,
    output reg MemWrite,
    output reg ALUSrc,
    output reg RegDst,
    output reg RegWrite,
    output reg Jump,
    output reg PCSrc,
    output reg [3:0] ALUControl,
    output reg isJR    // JR output as reg
);

reg Branch, BNE;

always @(*) begin
    // Default values
    {RegWrite, RegDst, ALUSrc, Branch, MemWrite, MemtoReg, Jump, BNE} = 8'b0;
    ALUControl = 4'b0000;
    isJR = 0;

    case (Opcode)
        6'b000000: begin // R-type
            RegWrite = 1;
            RegDst = 1;
            case (Func)
                6'b100000: ALUControl = 4'b0000; // ADD
                6'b100010: ALUControl = 4'b0001; // SUB
                6'b100100: ALUControl = 4'b0010; // AND
                6'b100101: ALUControl = 4'b0011; // OR
                6'b001000: begin // JR
                    isJR = 1;
                end
            endcase
        end
    endcase
end
```

```

        RegWrite = 0;
        Jump = 1;
    end
    // New additions for min/max
    6'b101100: ALUControl = 4'b1111; // MIN
    6'b101101: ALUControl = 4'b1110; // MAX
    default: ALUControl = 4'b0000;
endcase
end

6'b000011: begin // JAL
    Jump = 1;
    RegWrite = 1;
end

6'b100011: begin // LW
    MemtoReg = 1;
    RegWrite = 1;
    ALUSrc = 1;
    ALUControl = 4'b0000; // ADD
end

6'b101011: begin // SW
    MemWrite = 1;
    ALUSrc = 1;
    ALUControl = 4'b0000; // ADD
end

6'b001111: begin // LUI
    ALUSrc = 1;
    RegWrite = 1;
    ALUControl = 4'b0000;
end

6'b000100: begin // BEQ
    Branch = 1;
    ALUControl = 4'b0001; // SUB
end

6'b000101: begin // BNE
    Branch = 1;
    BNE = 1;

```

```
    ALUControl = 4'b0001; // SUB  
end
```

```
6'b000010: Jump = 1; // J
```

```
default: begin
```

```
    RegWrite = 0;
```

```
    RegDst = 0;
```

```
    ALUSrc = 0;
```

```
    Branch = 0;
```

```
    MemWrite = 0;
```

```
    MemtoReg = 0;
```

```
    Jump = 0;
```

```
    BNE = 0;
```

```
    ALUControl = 4'b0000;
```

```
end
```

```
endcase
```

```
    PCSrc = Branch & (Zero ^ BNE);
```

```
end
```

```
endmodule
```

Updated Datapath.v

```
// file: Datapath.v

`include "adder.v"
`include "alu32.v"
`include "flopr_param.v"
`include "mux2.v"
`include "mux4.v"
`include "regfile32.v"
`include "signext.v"
`include "sl2.v"

`timescale 1ns/1ns

module Datapath(input clk,
               input reset,
               input RegDst,
               input RegWrite,
               input ALUSrc,
               input Jump,
               input isJR,
               input MemtoReg,
               input PCSrc,
               input [3:0] ALUControl,
               input [31:0] ReadData,
               input [31:0] Instr,
               output [31:0] PC,
               output ZeroFlag,
               output [31:0] datatwo,
               output [31:0] ALUResult);

wire [31:0] PCNext, PCplus4, PCbeforeBranch, PCBranch;
wire [31:0] extendedimm, extendedimmafter, MUXresult, dataone, aluop2;
wire [4:0] writereg;
wire [31:0] jumpTarget;
wire [31:0] luiimm;

// LUI is a special case of immediate. It loads the immediate into the upper 16 bits.
assign luiimm = {Instr[15:0], 16'b0};
```



```

// PC Logic (Updated for JR/JAL)
flopr_param #(32) PCregister(clk, reset, PCNext, PC);
adder #(32) pcadd4(PC, 32'd4, PCplus4);
slt2 shifteradd2(extendedimm, extendedimmafter);
adder #(32) pcaddsigned(extendedimmafter, PCplus4, PCbeforeBranch);
assign jumpTarget = isJR ? dataone : {PCplus4[31:28], Instr[25:0], 2'b00};

mux2 #(32) branchmux(PCplus4, PCbeforeBranch, PCSrc, PCBranch);
mux2 #(32) jumpmux(PCBranch, jumpTarget, (Jump || isJR), PCNext);

// Register File (Updated for JAL)
mux2 #(5) writeopmux(
    Instr[20:16],
    (Jump && !isJR) ? 5'b11111 : Instr[15:11],
    RegDst,
    writereg
);

registerfile32 RF(
    clk, RegWrite, reset,
    Instr[25:21], Instr[20:16], writereg,
    (Jump && !isJR) ? PCplus4 : MUXresult,
    dataone, datatwo
);

// ALU

alu32 alucomp(dataone, aluop2, ALUControl, Instr[10:6], ALUResult, ZeroFlag);
signext immextention(Instr[15:0], extendedimm);

// New mux to select between sign-extended immediate and LUI immediate
mux2 #(32) aluop2sel(datatwo, (Instr[31:26] == 6'b001111) ? luiimm : extendedimm,
    ALUSrc, aluop2);

endmodule

```

Updated ALU32.v

```
`timescale 1ns/1ns

module alu32( input [31:0] a,
             input [31:0] b,
             input [3:0] f,
             input [4:0] shamt,
             output reg [31:0] y,
             output reg zero);

always @ (*) begin
    case (f)
        4'b0000: y = a + b; // ADD
        4'b0001: y = a - b; // SUB
        4'b0010: y = a & b; // AND
        4'b0011: y = a | b; // OR
        4'b0100: y = a ^ b; // XOR
        4'b0101: y = b << shamt; // SLL
        4'b0110: y = b >> shamt; // SRL
        4'b0111: y = $signed($signed(b) >>> shamt); // SRA
        4'b1000: y = $signed(a) < $signed(b) ? 32'd1 : 32'd0; // SLT
        4'b1001: y = a < b ? 32'd1 : 32'd0; // SLTU
        4'b1010: y = ~(a | b); // NOR
        4'b1011: y = b << a; // SLLV
        4'b1100: y = b >> a; // SRLV
        4'b1101: y = $signed($signed(b) >>> a); // SRAV
        4'b1110: y = a < b ? 32'd1 : 32'd0; // MIN_TEST
        4'b1111: y = a > b ? 32'd1 : 32'd0; // MAX_TEST
        default: y = 32'bx;
    endcase
    zero = (y==32'b0);
end
endmodule
```

Updated Assembler

```
.text
main:
    li $s0, 2
    li $s1, 10
    li $s2, 0
    li $s3, 12
    li $s4, 10
    li $s5, 12
    li $s6, 12
    li $s7, 12
    li $t8, 12
    li $t9, 12

    # Store numbers in memory (addresses 0-36)
    sw $s0, 0($0)
    sw $s1, 4($0)
    sw $s2, 8($0)
    sw $s3, 12($0)
    sw $s4, 16($0)
    sw $s5, 20($0)
    sw $s6, 24($0)
    sw $s7, 28($0)
    sw $t8, 32($0)
    sw $t9, 36($0)

    # Call functions
    add $a0, $0, $0    # Memory base
    li $a1, 10        # Count
    jal MAX
    add $s0, $v0, $0

    add $a0, $0, $0
    li $a1, 10
    jal MIN
    add $s1, $v0, $0

    add $a0, $0, $0
    li $a1, 10
    jal MEAN
    add $s2, $v0, $0

    # Store results (addresses 40-48)
    sw $s0, 40($0)    # Maximum
    sw $s1, 44($0)    # Minimum
    sw $s2, 48($0)    # Mean

    li $v0, 10        # Exit syscall
    syscall
```

MAX:

```
lw $v0, 0($a0)    # Init max
li $t0, 1          # Counter
li $t1, 4          # Word size
```

max_loop:

```
slt $t2, $t0, $a1  # counter < count?
beq $t2, $0, max_end
```

```
# Calculate address: $a0 + ($t0 << 2)
sll $t3, $t0, 2    # counter * 4
add $t3, $a0, $t3   # current address
lw $t4, 0($t3)     # load element
```

```
slt $t5, $v0, $t4   # max < current?
beq $t5, $0, max_next
add $v0, $t4, $0    # update max
```

max_next:

```
add $t0, $t0, 1     # counter++
j max_loop
```

max_end:

```
jr $ra
```

MIN:

```
lw $v0, 0($a0)    # Init min
li $t0, 1          # Counter
li $t1, 4          # Word size
```

min_loop:

```
slt $t2, $t0, $a1  # counter < count?
beq $t2, $0, min_end
```

```
# Calculate address: $a0 + ($t0 << 2)
sll $t3, $t0, 2    # counter * 4
add $t3, $a0, $t3   # current address
lw $t4, 0($t3)     # load element
```

```
slt $t5, $t4, $v0   # current < min?
beq $t5, $0, min_next
add $v0, $t4, $0    # update min
```

min_next:

```
add $t0, $t0, 1     # counter++
j min_loop
```

min_end:

```
jr $ra
```

MEAN:

```
li $t0, 0      # Counter
li $v0, 0      # Sum
li $t1, 4      # Word size
```

sum_loop:

```
slt $t2, $t0, $a1 # counter < count?
beq $t2, $0, divide_sum

# Calculate address: $a0 + ($t0 << 2)
sll $t3, $t0, 2   # counter * 4
add $t3, $a0, $t3 # current address
lw $t4, 0($t3)    # load element

add $v0, $v0, $t4 # add to sum
add $t0, $t0, 1   # counter++
j sum_loop
```

divide_sum:

```
add $a0, $v0, $0 # Dividend (sum)
li $a1, 10       # Divisor (count)
li $v0, 0        # Quotient (mean)
```

divide_loop:

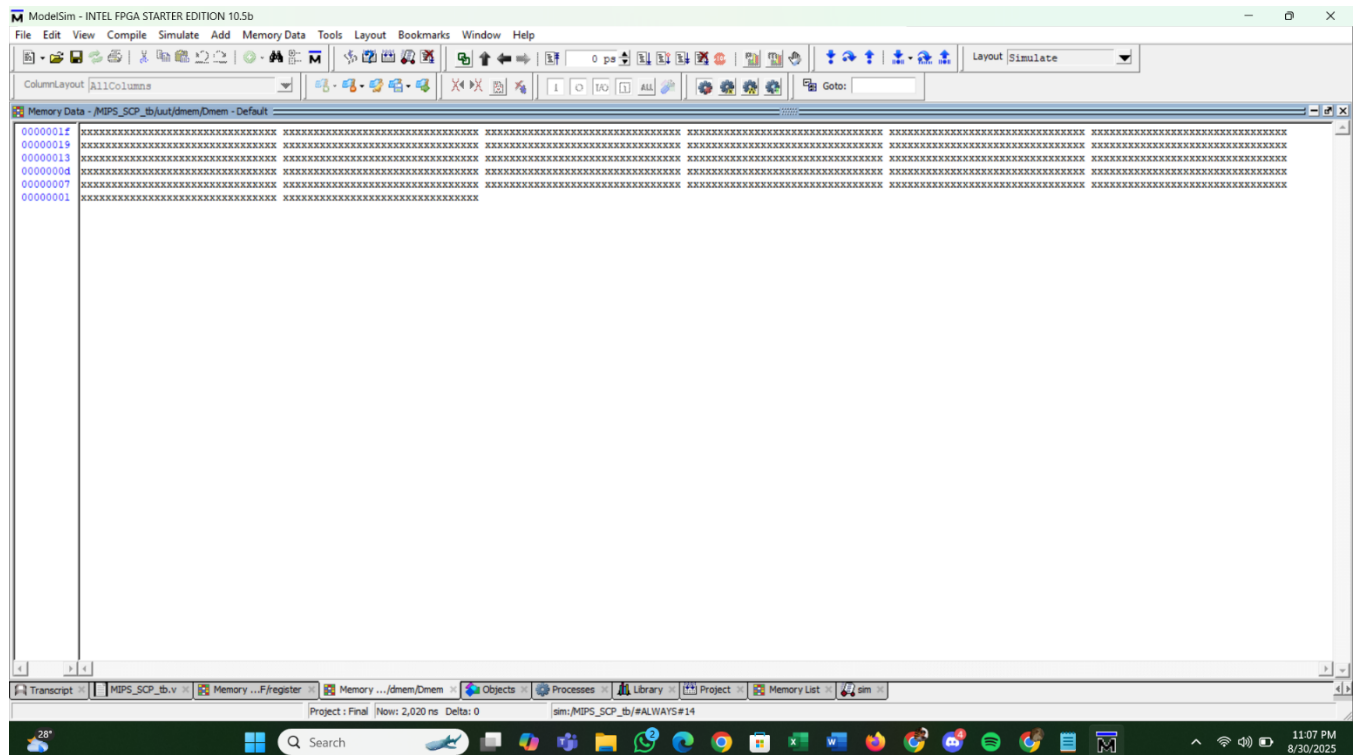
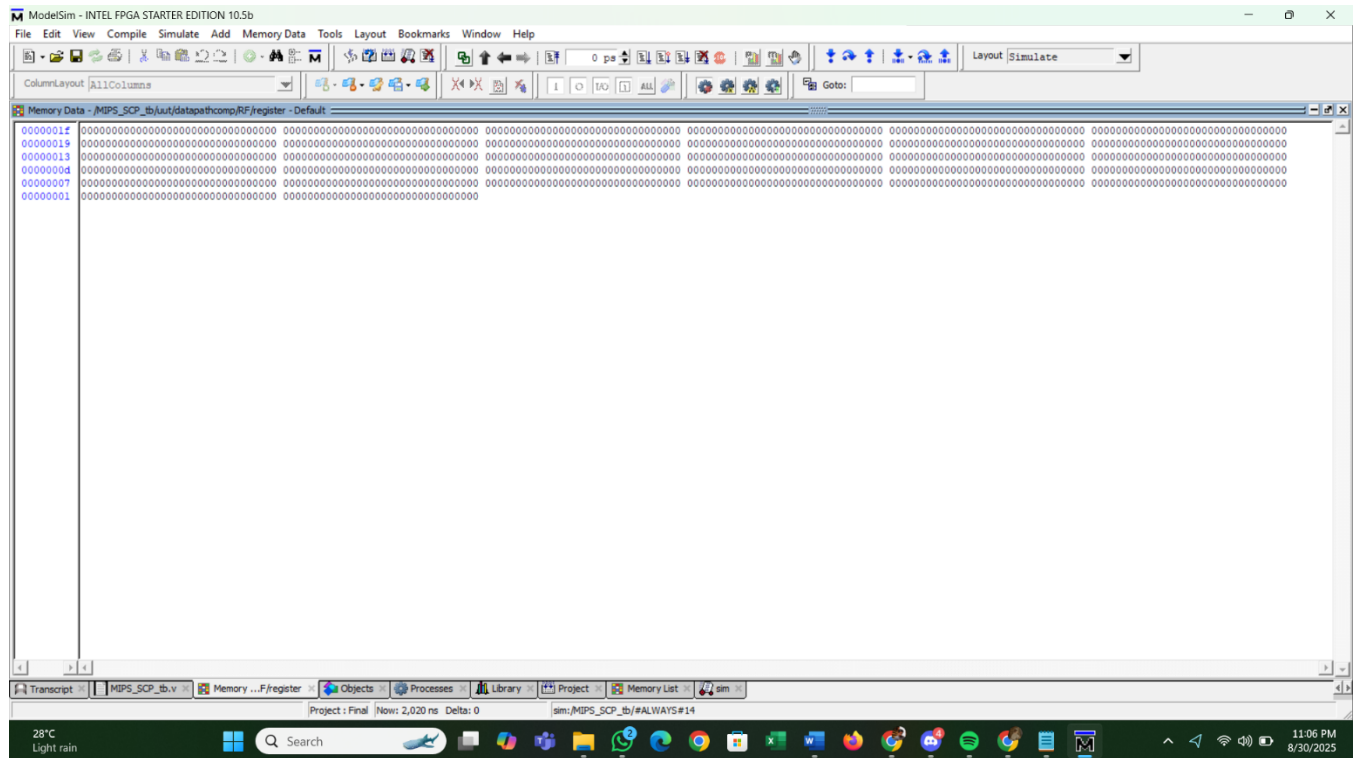
```
slt $t0, $a0, $a1 # Check if dividend < divisor
bne $t0, $0, divide_end

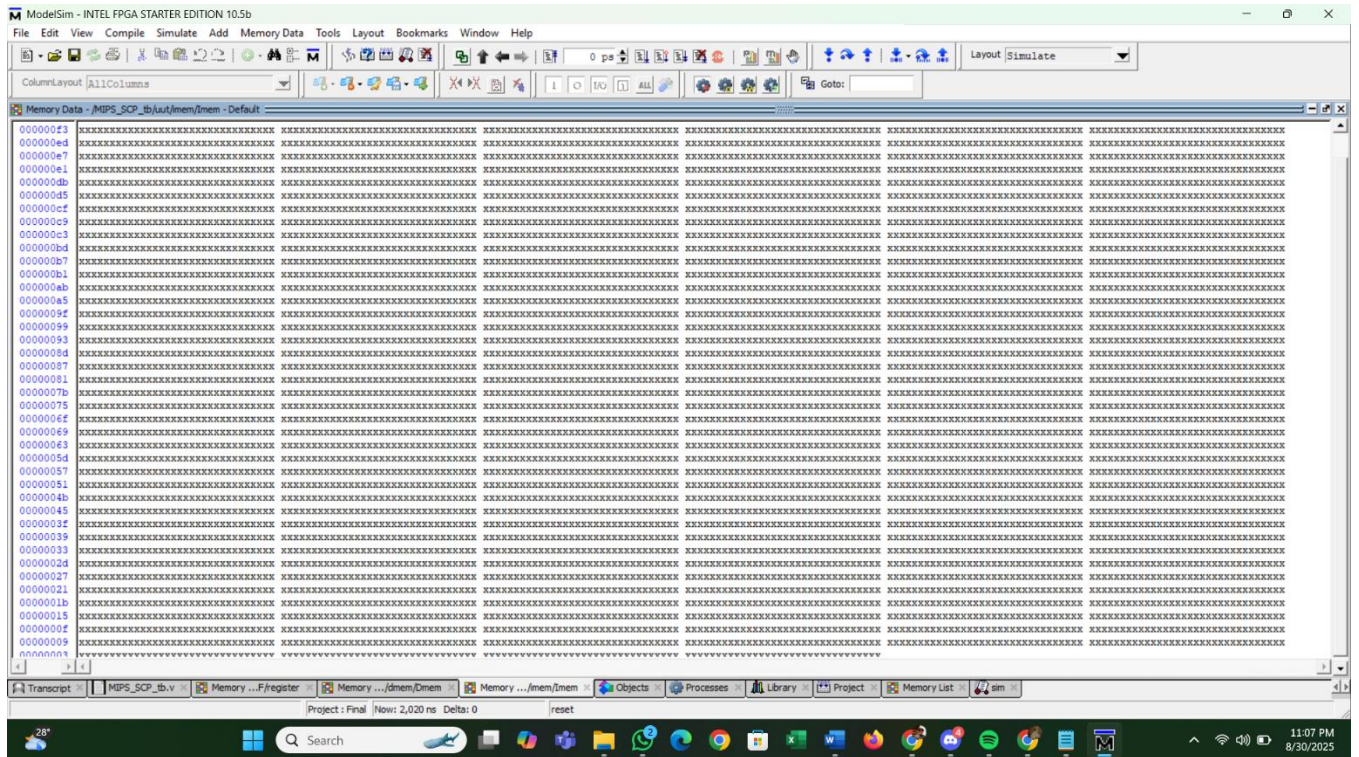
sub $a0, $a0, $a1 # Subtract divisor from dividend
add $v0, $v0, 1   # Increment quotient
j divide_loop
```

divide_end:

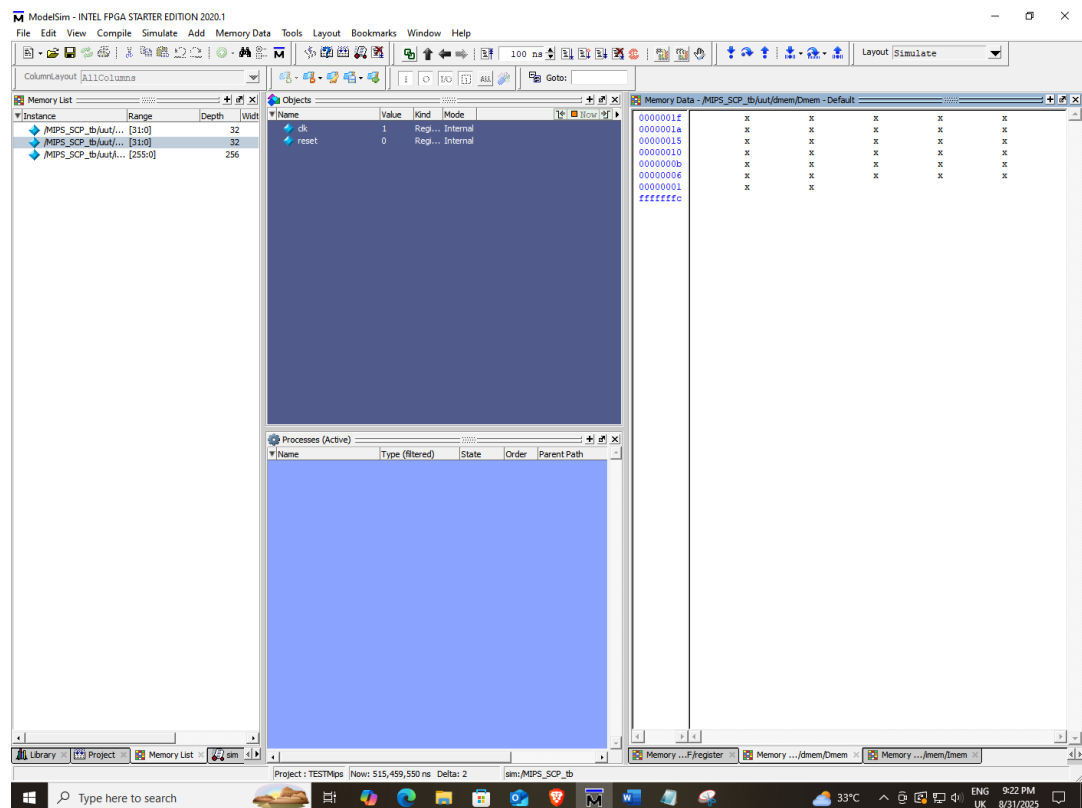
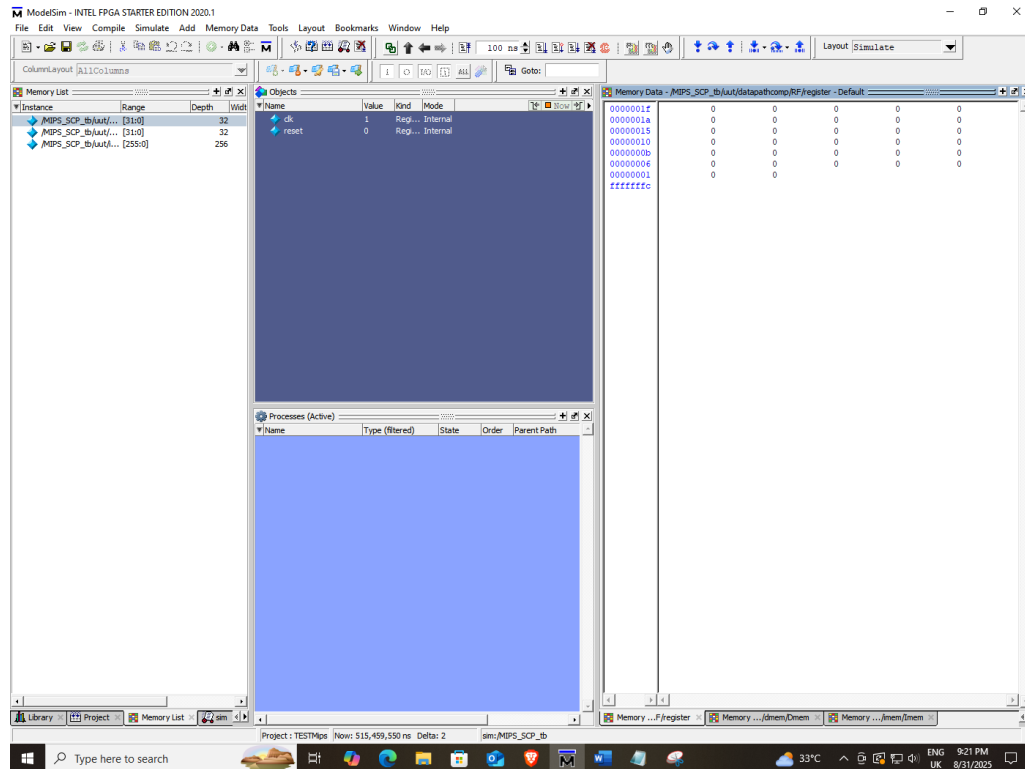
```
jr $ra
```

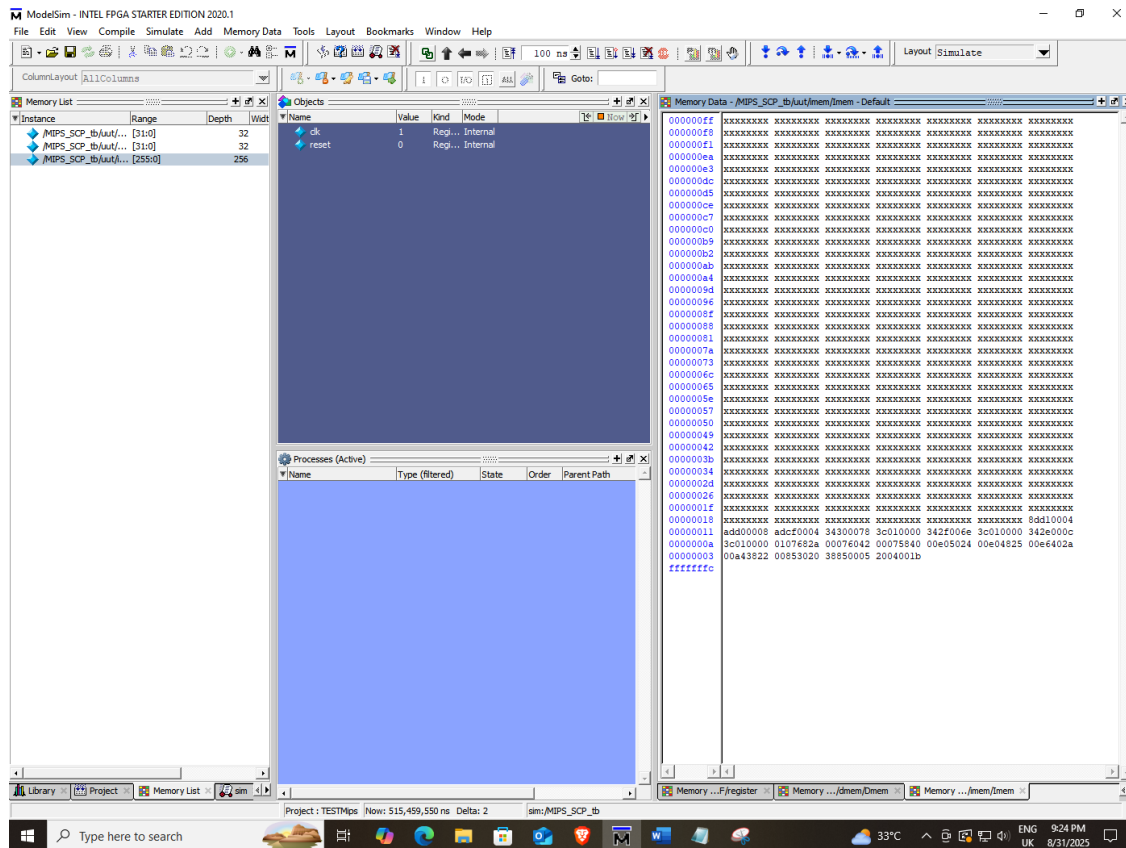
Memory List From the Base-Verilog MIPS CPU & OS:





Modified MIPS Verilog Simulations – Memory List:





MEMFILE.txt

```
0010000000000000100000000000000011011
001110001000010100000000000000101
00000000100001010011000000100000
00000000101001000011100000100010
00000000111001100100000000101010
0000000011100000100100000100101
00000000111000000101000000100100
0000000011101011011000001000000
000000000000001110110000001000010
000000000000001110110100000101010
00111100000000100000000000000000
00110100001011100000000000001100
00111100000000100000000000000000
001101000010111100000000001101110
00111100000000010000000000000000
001101000011000000000000001111000
10101101110011110000000000000100
10101101110100000000000000001000
10001101110100010000000000000100
```

Conclusion

In conclusion, this project successfully demonstrated the process of modifying a standard MIPS processor to include additional instructions that expand both its control flow and arithmetic capabilities. The incorporation of JAL and JR instructions enables robust subroutine management through efficient call and return operations, reflecting real-world compiler behavior. Additionally, the integration of custom arithmetic instructions MIN, MAX, and MEAN showcased how specialized logic can be embedded directly into the ALU, enabling the processor to perform more complex tasks without relying on software-level computation.

The modifications were implemented structurally within the Verilog HDL, ensuring hardware-level consistency, and tested thoroughly within the MIPS_332 simulation environment. Despite minor assembler constraints, the successful operation of each instruction under waveform inspection verifies the correctness of our design. This hands-on exercise enhanced our understanding of processor architecture, datapath extension, and instruction decoding. Moreover, it provided practical insight into how real-world processors evolve by expanding instruction sets for performance and functionality improvements.

Through this project, we have not only deepened our knowledge of digital design and MIPS architecture but also gained valuable experience in structured hardware development and debugging, equipping us with essential skills for advanced system-level hardware design in future endeavors.

